Repaso de Python

PROFESORA VICTORIA GUTIERREZ

“

Elementos
Básicos de Python
Fundamentos

Python

Python es un lenguaje que se desarrolla y mejora continuamente

Es importante tener a disposición la documentación y recursos disponibles online

- Comunidad Python Argentina: www.python.org.ar
- Documentación Oficial Python: www.docs.python.org

Python

Lenguaje, escritura y Sintaxis

- Las instrucciones del lenguaje se realizan mediante comandos simples. Instrucciones complejas pueden realizarse mediante varias simples

Ejemplos de instrucciones

And, or, as, del, if, for, from, continue, break, def, return, while, ...

Las instrucciones son palabras reservadas, no pueden ser utilizadas en el código con otro fin.

Python

Lenguaje, escritura y Sintaxis - Ejemplo

Programa
.py

```python
print("Hola a todos!")

print("Esto es una prueba")

edad = 20

if edad > 18:
    print("Es mayor de edad!")

print("FIN")
```

```
Hola a todos!
Esto es una prueba
Es mayor de edad!
FIN
[Finished in 0.8s]
```

Python

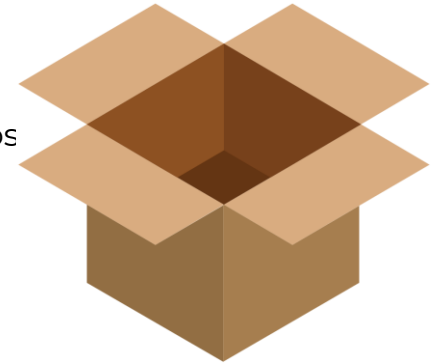
Lenguaje, escritura y Sintaxis

- La sintaxis de Python es extremadamente "limpia",
  no se requiere de ningún carácter que indique el final de una secuencia y los bloques se definen a través de
  la indentación del código

Indentación: Uso informática. Sangría de texto a la derecha por espacios o tabulaciones

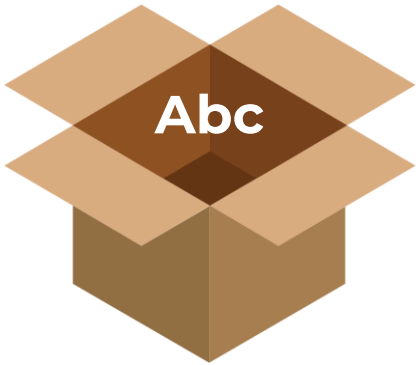
- **Variables**: Una variable es un sitio donde se guardan los datos para su posterior utilización

Como una caja, donde podemos almacenar valores
Siempre que necesitemos, hacemos referencia a la caja

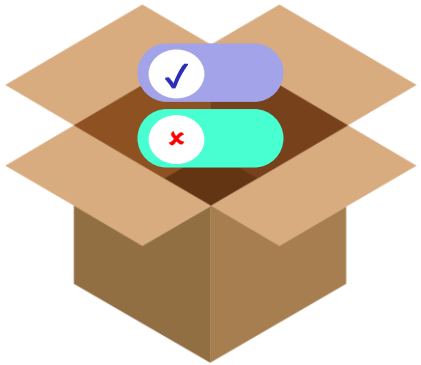


- El dato que almacena una variable debe ser de un tipo determinado. Por ejemplo, numérico, cadena de caracteres o strings, lógicos, etc.

Numérico



Strings



Lógicos

Python

Tipo de datos

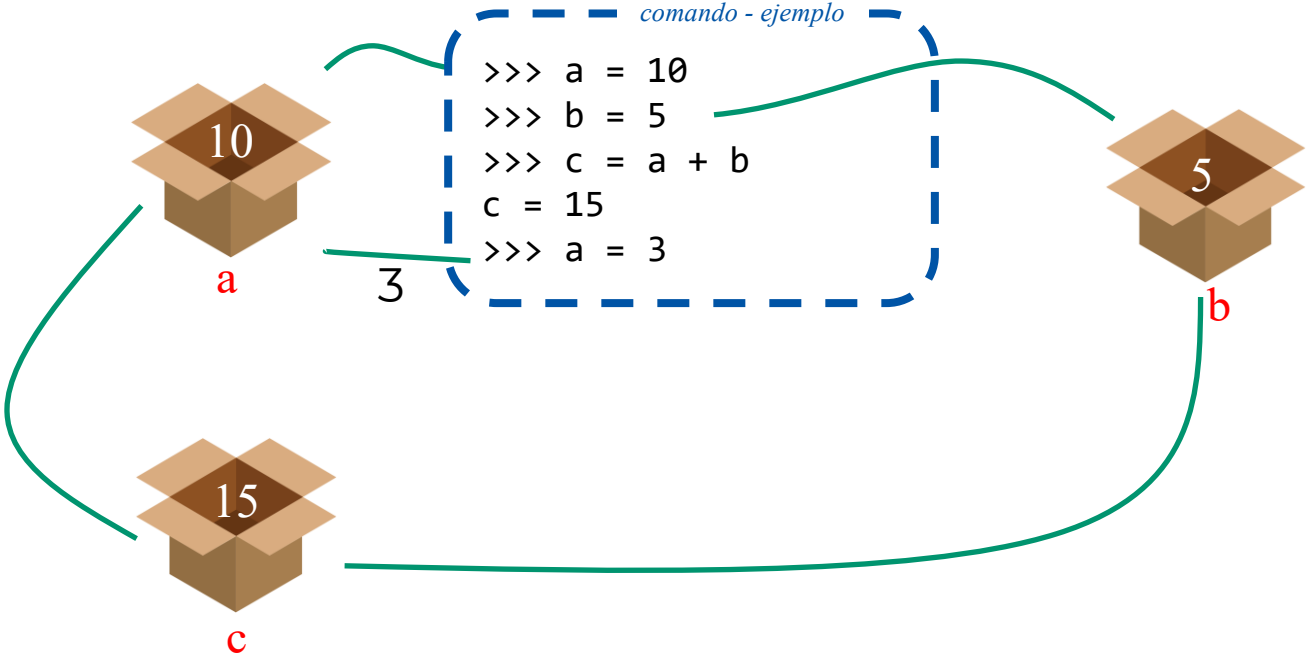
- Numéricos: enteros, reales
- Cadena de caracteres
- Valores booleanos

comando - ejemplo

```
num = 20            #Entero
num = 0.27          #Real
c = ”Hola Mundo”    #Cadena de caracteres
b = True            #Logico
```

Python

Tipo de datos - Ejemplos



Python

Operadores Aritméticos – (1/2)

Operador	**Descripción**	**Ejemplo**	**Resultado**
+	Suma	r = 3 + 2	r = 5
-	Resta	r= 4 - 7	r = -3
-	Negación	r = -7	r = -7
*	Multiplicación	r = 2 * 6	r = 12
**	Exponente	r = 2 ** 6	r = 64
/	División	r = 3.5 / 2	r = 1.75
//	División entera	r = 3.5 // 2	r = 1.0
%	Módulo	r = 7 % 2	r = 1

Python

Operadores Aritméticos – (2/2)

Operadores incrementar += decrementar -=

Operadores lógicos
and, **or**, y **not**

Operadores de comparación
distinto que **!=**
mayor que **>**
menor que **<**
mayor o igual **>=**
menor o igual **<=**

== se utiliza para comprobar que **dos valores son iguales**

= se utiliza para la asignación de valores a variables

Python

Comentarios

- Los comentarios en Python sirven para documentar el programa

Pueden ser:

- De una sola línea (#) al principio de la misma
- De múltiples líneas (""") al principio y al final

comando - ejemplo

```
#Esto es un comentario

"""Esto también es un
   comentario pero es
   multilínea ..."""
```

Python

Identificadores reservados - Resumen

- Los utiliza el lenguaje con propósitos particulares como parte de las instrucciones
- No pueden utilizarse como variables, tienen objetivos específicos.

```
and       del       from      not       while
as        elif      global    or        with
assert    else      if        pass      yield
break     except    import    print
class     exec      in        raise
continue  finally   is        return
def       for       lambda    try
```

And, or, not

Operadores lógicos en Python.

- And es verdadero solo si ambos operandos son verdaderos.
- Or es verdadero si al menos uno de los operandos lo es.
- Not invierte el valor de verdad.

La tabla de verdad se ejemplifica a continuación.

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Tabla de verdad AND

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Tabla de verdad OR

A	Not A
True	False
False	True

Tabla de verdad NOT

comando - ejemplo

```
>>> True and False
False
>>> True or False
True
>>> not False
True
```

Python

Identificadores reservados – Palabras Claves – (2/7)

True, False

Valores de verdad en Python. Utilizados en comparaciones y condicionales

comando - ejemplo

```
>>> 1 == 1
True
>>> 5 > 3
True
>>> True or False
True
>>> 10 <= 1
False
>>> 3 > 7
False
>>> True and False
False
>>> True == 1
True
>>> False == 0
True
>>> True + True
2
```

None

Es una constante especial que representa la ausencia o un valor NULO.

comando - ejemplo

```
>>> None == 0
False
>>> None == []
False
>>> None == False
False
>>> x = None
>>> y = None
>>> x == y
True
```

Import, from

import es utilizado para importar (incluir) otras librerías y sus funcionalidades.
Con la keyword *from* solo se importan las funciones especificadas de dicha **librería**

comando - ejemplo

```
import openpyxl
from openpyxl import load_workbook
```

In

Es utilizado para evaluar si una secuencia de valores (enteros, reales, strings, etc) contiene o no un valor. Retorna True si el valor esta contenido en la lista de valores, caso contrario False

comando - ejemplo

```
>>> a = [1, 2, 3, 4, 5]
>>> 5 in a
True
>>> 10 in a
False
```

comando - ejemplo

```
for i in 'hello':
    print(i)
```

```
h
e
l
l
o
```

def

Keyword utilizada para la definición de funciones.
Una función es un bloque de acciones relacionadas entre si, las cuales juntas realizar una secuencia de pasos hasta finalizar en un resultado. Es útil para distribuir el código en pequeñas porciones o bien cuando estas porciones son repetitivas.

comando - ejemplo

```
def function_name(parameters):
    a = 10 + 20
    c = a + 10
    return c
```

return

Return es utilizado para devolver un valor en una función al momento de su salida o fin.

comando - ejemplo

```
def func_return():
    a = 10
    return a

print(func_return())
```

If, else, elif

If-else son identificadores utilizados para indicar bloques de código condicional o toma de decisiones.
Cuando se requiere evaluar una condición de prueba (donde la misma debe retornar verdadero o falso) puede utilizarse dichas sentencias.
Elif es una abreviatura para else-if para condicionales anidados.

comando - ejemplo

```
If condición:
   Verdadero
else:
   Falso
```

comando - ejemplo

```
If var > 5:
   print("var es mayor que 5")
else:
   print("var es menor que 5")
```

for

for es utilizado para iterar sobre una secuencia de valores. Generalmente es utilizado cuando se conoce la cantidad de números por los cuales iterar.

comando - ejemplo

```
for i in [1,2,3]:
    print(i)
```

Break, continue

Son utilizadas dentro de los ciclos for y while para cortar el curso normal del mismo. Break para el ciclo completamente, mientras que continue salta las hasta el final de las líneas del ciclo actual.

comando - ejemplo

```
for i in range(1,7):
    if i == 5:
        break
    print(i)
```

```
1
2
3
4
```

comando - ejemplo

```
for i in range(1,7):
    if i == 5:
        continue
    print(i)
```

```
1
2
3
4
6
7
```

while

While es utilizado para iterar sobre una ciclo, generalmente desconocido en principio. Itera hasta que se cumple la condición.

comando - ejemplo

```
i = 5
while(i):
    print(i)
    i = i - 1
```

```
5
4
3
2
1
```

Cadena de Caracteres

Se definen por comillas simples (‘) o comillas dobles (“): ‘hola’ es lo mismo que “hola”

Caracteres Especiales

\n: nueva línea

\t: tabulador horizontal

Métodos con cadenas

lower(): devuelve la cadena en minúsculas
upper(): devuelve la cadena en mayúsculas
replace(): reemplaza una cadena con otra cadena
split(): divide la cadena en subcadenas
count(): Devuelve el número de veces que se encuentra un valor especifio en una cadena
find(): Busca en la cadena un valor especifico y devuelve la posición donde se encontró

Diccionarios

Un diccionario es una colección desordenada, modificable e indexada. En Python, los diccionarios se escriben entre llaves y tienen:

CLAVES

VALORES

```
diccionario = {
    “nombre": “Juan",
    “apellido": “Perez",
    “edad": 23
}
```

Enumerate

El método enumerate() agrega un contador a una lista y lo devuelve en forma de tupla.

enumerate(lista, inicio = número)

Lista o similar
inicio (opcional): enumerate() comienza a contar desde un número, si este se omite, se toma como inicio 0.

comando - ejemplo

```
alumnos = ['Juan', 'Micaela', 'Samuel']
enumerateAlumnos = enumerate(alumnos)

# lo convierto a una lista
print(list(enumerateAlumnos))
```

Funciones

Una función es un bloque de código que solo se ejecuta cuando se lo invoca para realizar alguna tarea.
En general, devuelven o retornan un resultado

Crear una función

Llamar a una función

comando - ejemplo

```
def mi_funcion(x, y, z):
        resultado = x + y + z
        return resultado

mi_funcion(x = 10, y = 20, z = 30)
mi_funcion(10, 20, 30)
```

Desde la perspectiva de una función

Un **parámetro** es la variable que se encuentra entre paréntesis en la definición de la función.

Un **argumento** es el valor que se envía a la función cuando se la llama.

Argumentos
Los argumentos se especifican dentro de los paréntesis.

¿Argumentos o Parámetros?
Se pueden decir que son lo mismo: información que se envía a una función (en forma de variable)

Funciones

Una función es un bloque de código que solo se ejecuta cuando se lo invoca para realizar alguna tarea.
En general, devuelven o retornan un resultado

comando - ejemplo

```
def mi_funcion(x, y, z):
        resultado = x + y + z
        return resultado
```

En el cuerpo de la función se define operaciones. **Estas sólo tienen validez dentro de la función**

La sentencia def indica que se va a definir una función

La sentencia return indica que lo que la función retornará

Funciones

Una función es un bloque de código que solo se ejecuta cuando se lo invoca para realizar alguna tarea. En general, devuelven o retornan un resultado

comando - ejemplo

```
def mi_funcion(x, y, z):
        resultado = x + y + z
        return resultado

mi_funcion(x = 10, y = 20, z = 30)
mi_funcion(10, 20, 30)
```

Los parámetros se colocan entre paréntesis. Es buena práctica nombrarlos (asignarlos en el momento)

Si los parámetros no son nombrados al momento de invocar la función, se asume que estos respetan el orden de definición

comando - ejemplo

```
mi_funcion(x = 10, y = 20, z = 30)
mi_funcion(10, 20, 30)
mi_funcion(30, -9, 0)
mi_funcion(z = 30, x = 10, y = 20)
```

Si se quiere cambiar el orden, deben nombrarse.